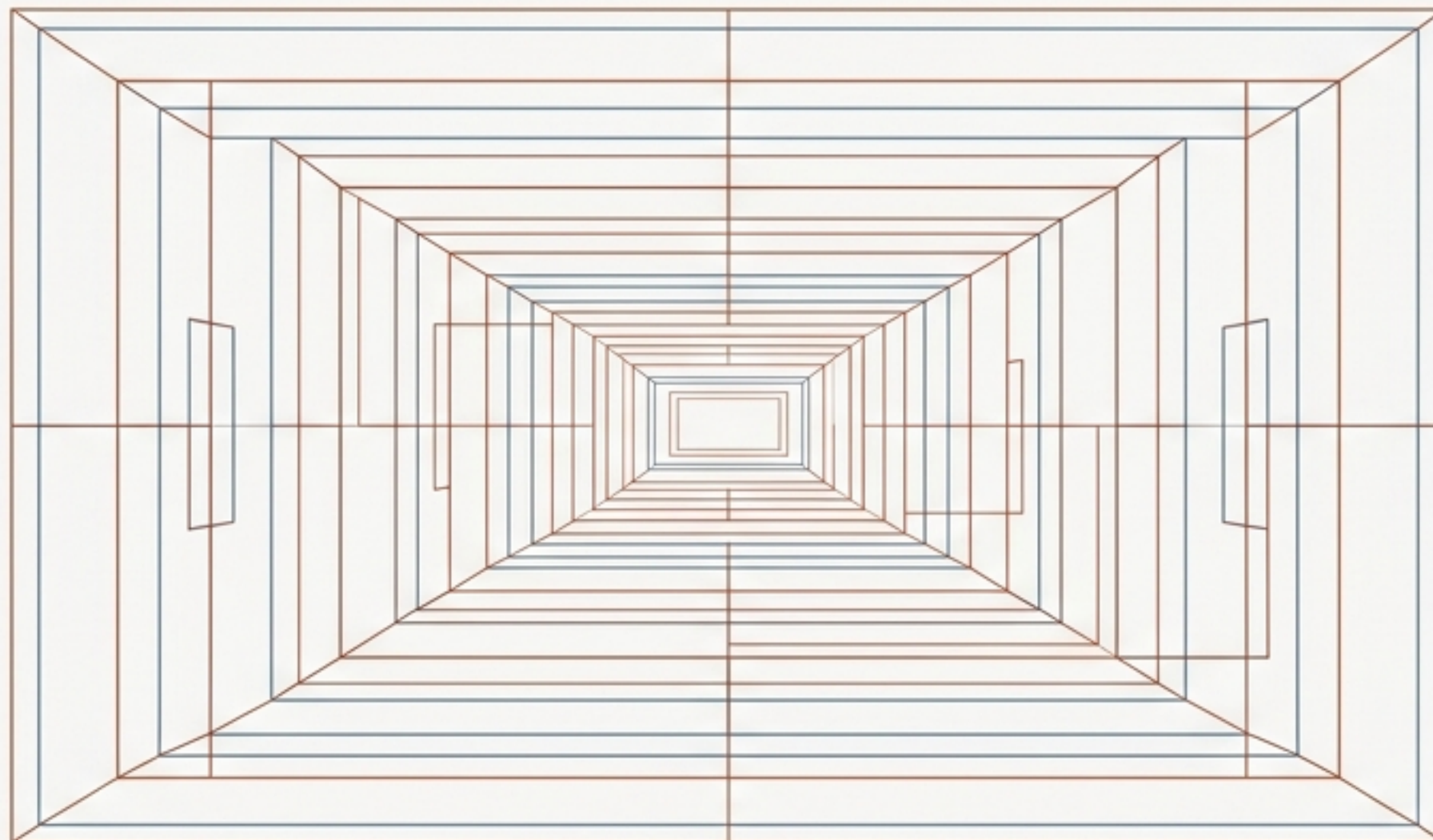
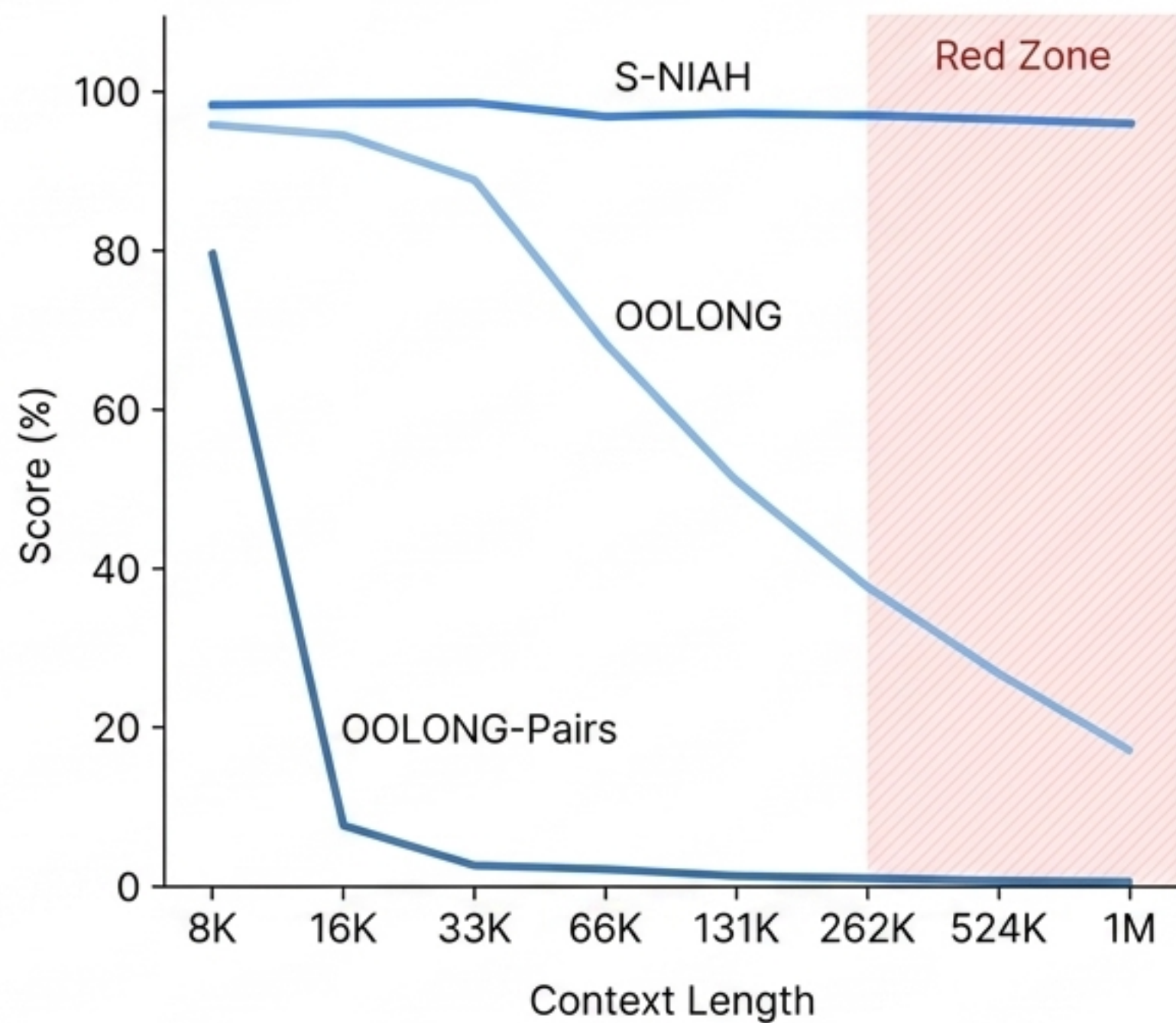




Recursive Language Models: ทะลุขีดจำกัด Context Window สู่ความยาวระดับอนันต์

กระบวนการใหม่ของการขยาย Inference-Time Compute
ผ่าน External REPL Environment และ Symbolic Recursion

MIT CSAIL Research Synthesis



ภาวะบริบทเสื่อมถอย (Context Rot) เมื่อความยาวและความซับซ้อน ทำลายประสิทธิภาพ



ปัญหาหลัก: Frontier Models มีข้อจำกัดด้าน Context Window แม้จะรองรับได้ถึง 1M+ Token แต่ความแม่นยำจะลดลงอย่างรุนแรงเมื่อบริบทยาวขึ้น



ตัวแปรความซับซ้อน: ยิ่งงานมีความซับซ้อนระดับ Quadratic (เช่น การจับคู่ข้อมูลใน OOLONG-Pairs) ประสิทธิภาพจะพังทลายลงจนเหลือเพียง 0.1% F1 Score แม้จะอยู่ใน Context Limit ก็ตาม

ข้อบกพร่องของสถาปัตยกรรมเดิม: การอัดเยียดข้อมูลลงในโมเดล



Context Compaction (การบีบอัด)

การสรุปข้อมูลซ้ำๆ ทำให้สูญเสียรายละเอียดเชิงลึก (Fine-grained details) ที่จำเป็นต่องานวิเคราะห์ที่ซับซ้อน

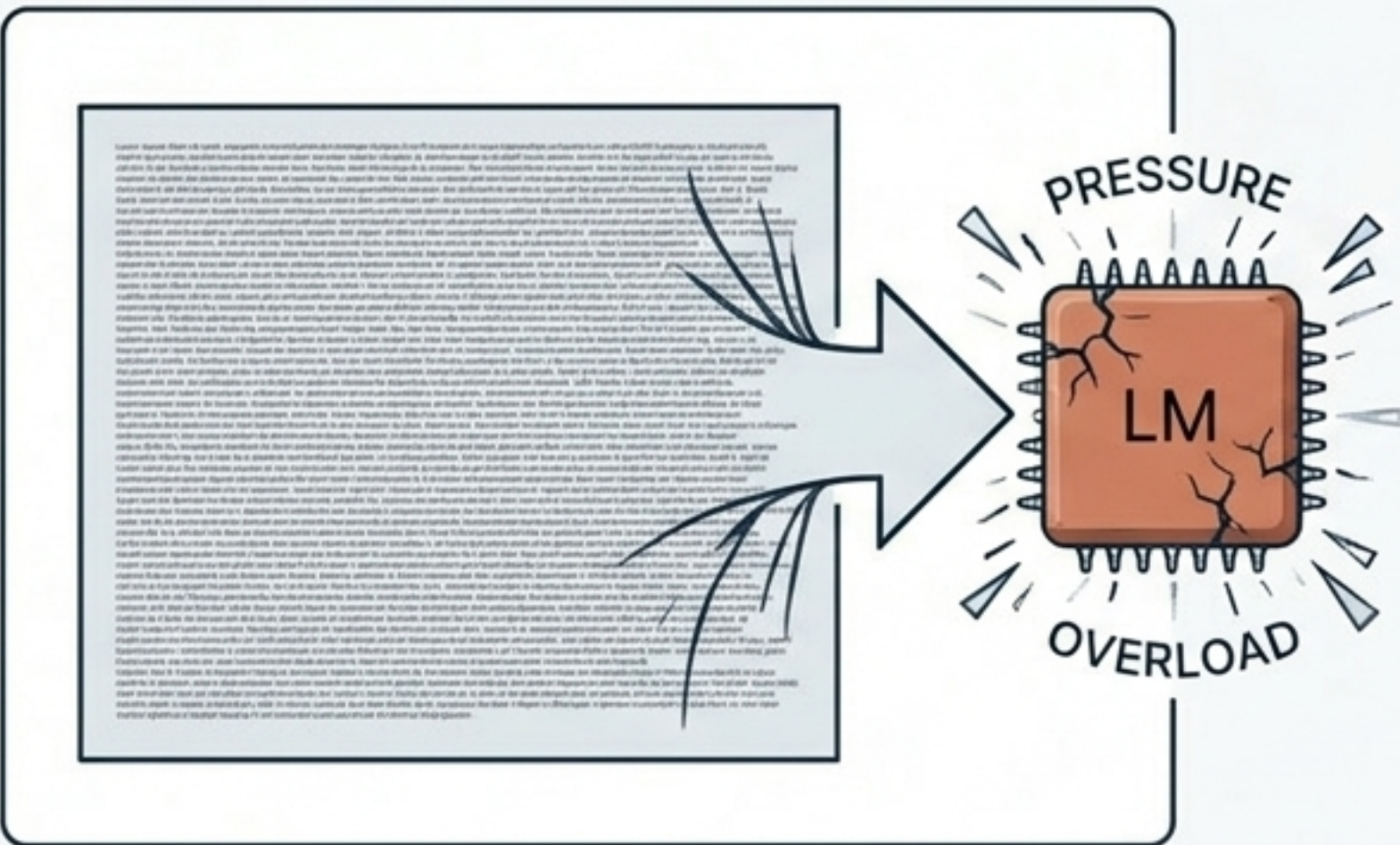


CodeAct / Retrieval (การดึงข้อมูล)

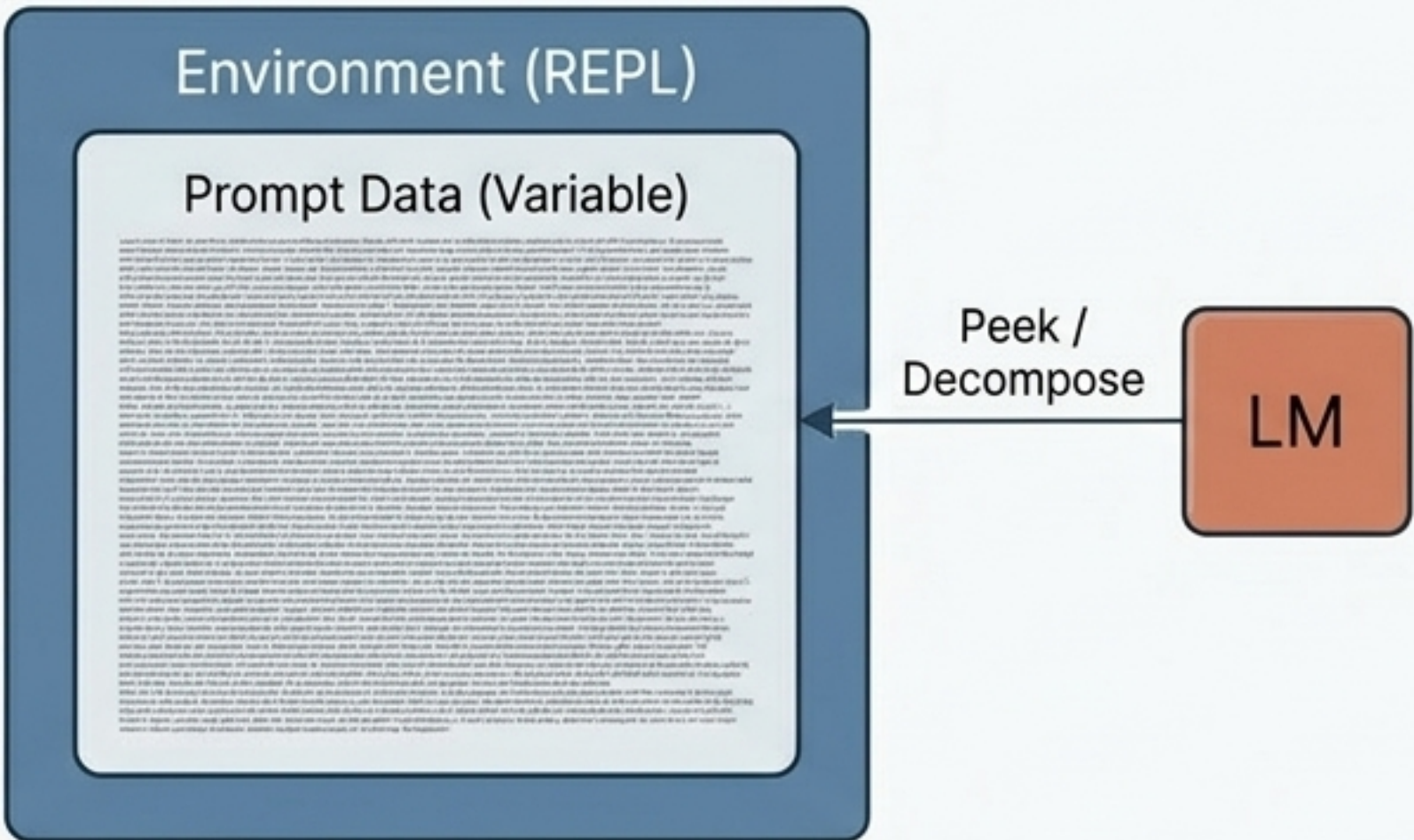
โมเดลยังคงพยายามนำข้อมูลทั้งหมดเข้าไปประมวลผลใน Context Window เริ่มต้น ทำให้ชนขีดจำกัดเดิมอย่างหลีกเลี่ยงไม่ได้

การเปลี่ยนกระบวนการ: ย้าย Prompt ออกสู่พื้นที่ทำงานภายนอก

แบบดั้งเดิม (ยัดข้อมูลเข้าโมเดล)



สถาปัตยกรรม RLM (พื้นที่ทำงาน)

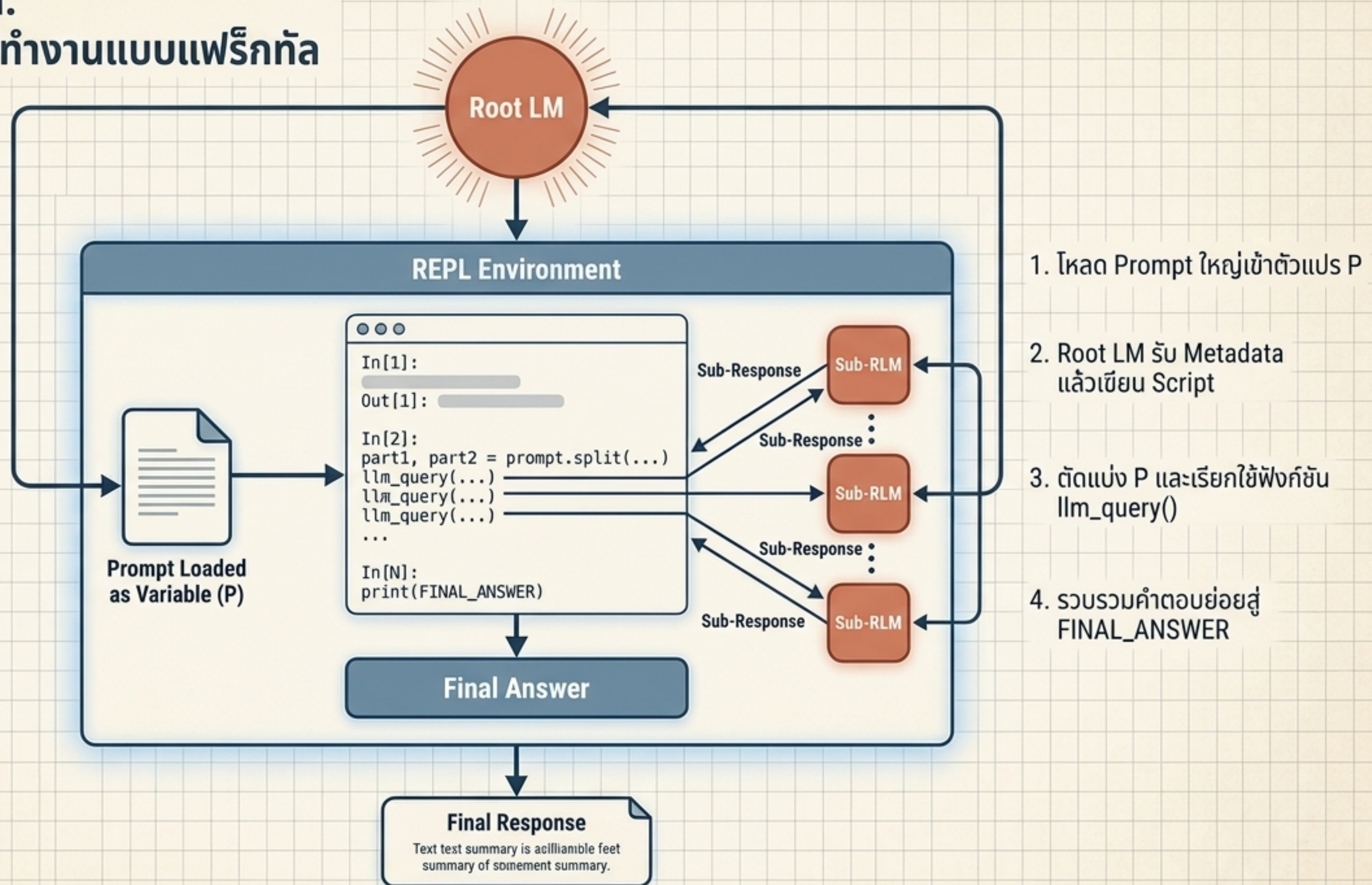


ไม่ป้อนข้อมูลโดยตรง: RLM ไม่ป้อน Prompt ขนาดยักษ์เข้าสู่ Neural Network

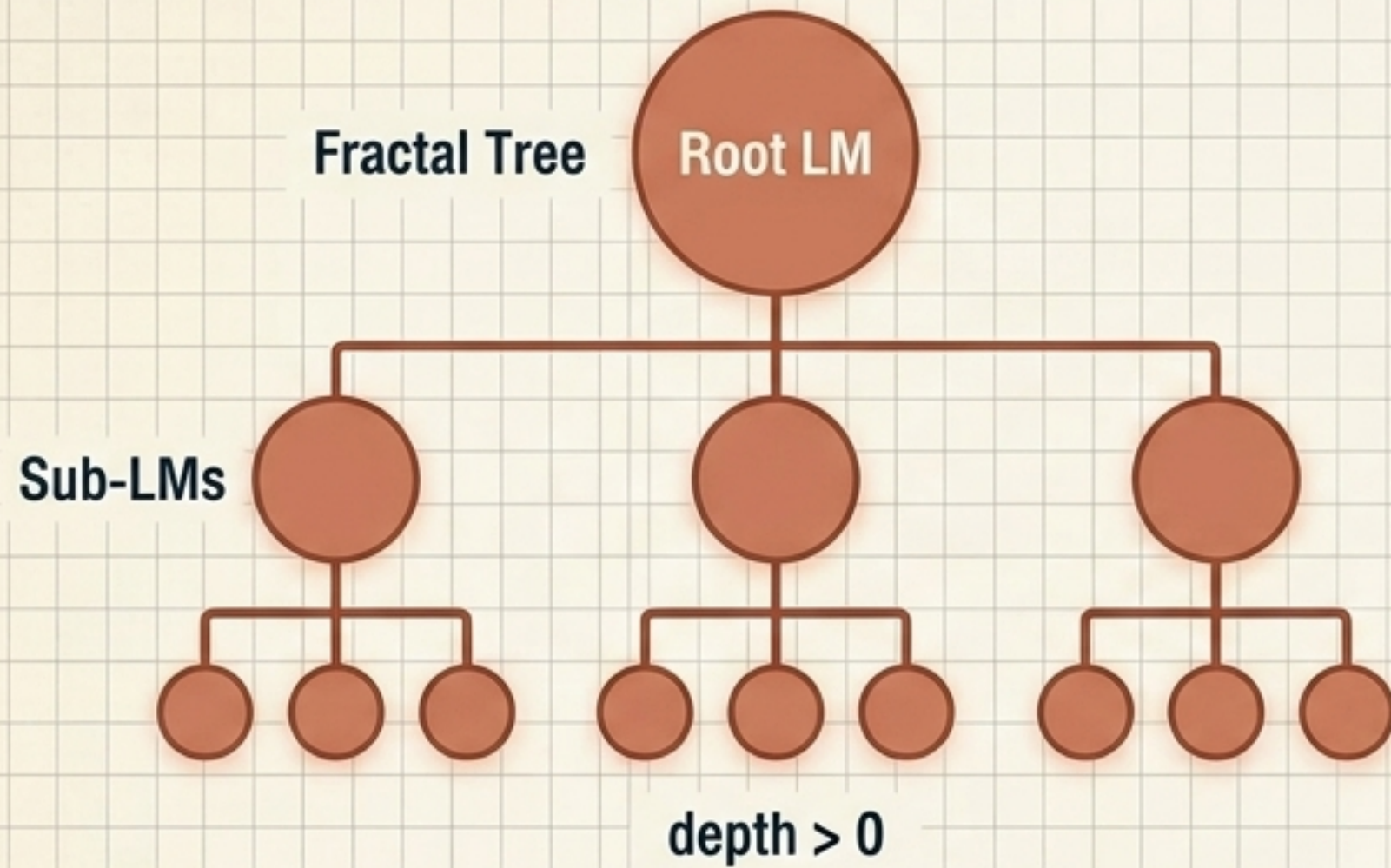
Prompt คือวัตถุภายนอก: เก็บข้อมูลทั้งหมดเป็นตัวแปร (Variable) ใน REPL

ทะลุขีดจำกัด: โมเดลรับรู้เพียง Metadata และเขียนโค้ดดึงข้อมูลมาวิเคราะห์

กายวิภาคของ RLM: สถาปัตยกรรมพื้นที่ทำงานแบบแฟร็กทัล



หัวใจสำคัญ: การเรียกซ้ำเชิงสัญลักษณ์ (Symbolic Recursion)



```
for chunk in data:  
    sub_response = llm_query(chunk)  
    aggregate.append(sub_response)
```

ความต่างที่แท้จริงไม่ใช่แค่การรันโค้ด
แต่คือความสามารถในการสร้างโปรแกรมที่มี Loop
เพื่อสร้าง Sub-LMs แทนแขนงออกไปจัดการข้อมูลย่อย

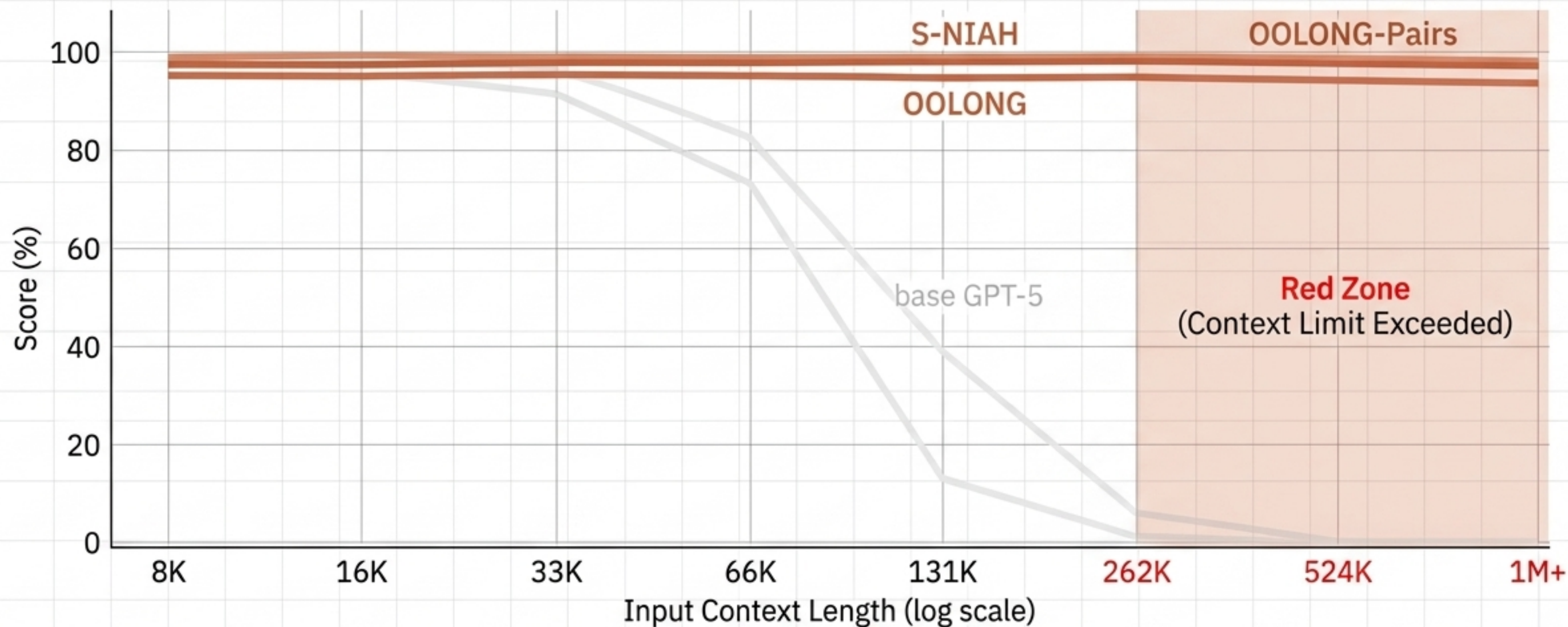
รองรับการวิเคราะห์ระดับเส้นตรงและควอดราติก
(Linear & Quadratic) ได้อย่างสมบูรณ์
โดยไม่ทำให้ Context ของ Root LM ล้น

ขีดความสามารถเชิงสถาปัตยกรรม: RLM vs. Alternatives

ความสามารถ	Base LLM	Compaction	CodeAct	RLM (Depth > 0)
Context Limit	Bounded	Infinite	Bounded	Infinite
Information Preservation	✔ Yes	✖ No	✔ Yes	✔ Yes
Semantic Transformation	✔ Yes	✖ No	Partial	✔ Yes
Symbolic Recursion	✖ No	✖ No	✖ No	✔ Yes

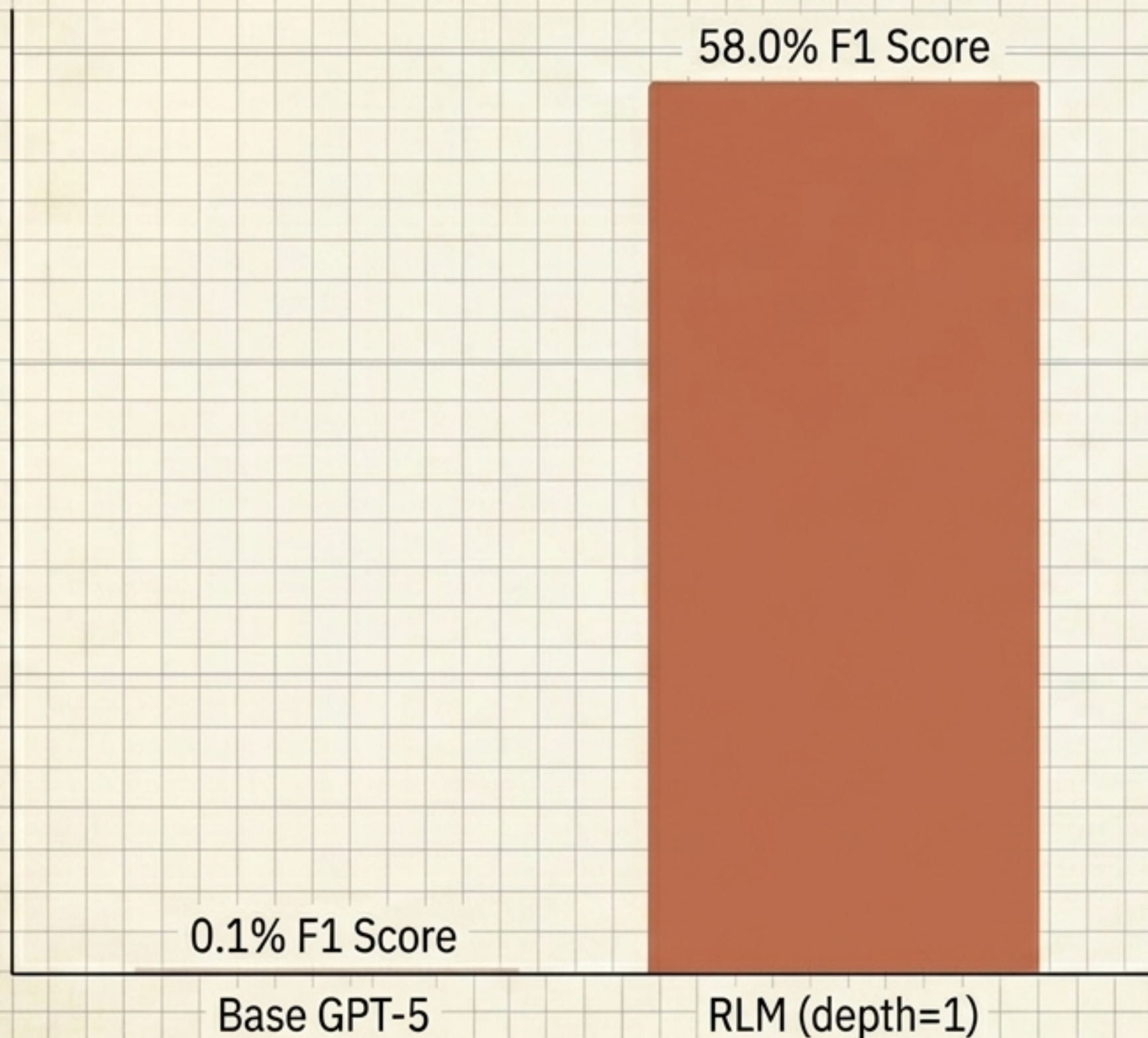
Takeaway: RLMs เป็นเฟรมเวิร์กเดียวที่ผสานการจัดการข้อมูลขนาดยักษ์ (Infinite Context) เข้ากับการวิเคราะห์เชิงลึกที่ต้องเข้าถึงข้อมูลแบบหนาแน่นได้อย่างไร้รอยต่อ

ทลายอุปสรรค: เสถียรภาพสมบูรณ์ข้ามขีดจำกัด Context



- **Scaling the Unscalable:** ในขณะที่ GPT-5 ปกติมีประสิทธิภาพลดลงอย่างรวดเร็ว RLM สามารถรักษาระดับความแม่นยำได้คงที่ข้ามขีดจำกัด Context Window
- รองรับข้อมูลยาวมหาศาลระดับ 1M+ Token อย่างแม่นยำ โดยไม่เกิดภาวะบริบทเสื่อมถอย

การจัดการข้อมูลที่มีความหนาแน่นสูง (Extreme Information Density)



โจทย์: OOLONG-Pairs

ความซับซ้อนระดับ Quadratic
ที่ต้องจับคู่ข้อมูลทุกชิ้นใน Prompt
ซึ่งโมเดลปกติล้มเหลวโดยสิ้นเชิง

ความได้เปรียบของ RLM

ในงานที่คำตอบขึ้นอยู่กับข้อมูลทุก
บรรทัด RLM ใช้ Recursive Sub-calls
ในการแบ่งนอกรแบ่งย่อยและเอาชนะ
ความซับซ้อนนี้ได้อย่างแม่นยำ

ทลายความเชื่อเรื่องต้นทุน: ประสิทธิภาพสูงเทียบเท่าต้นทุนเดิม

The Cost Contrast

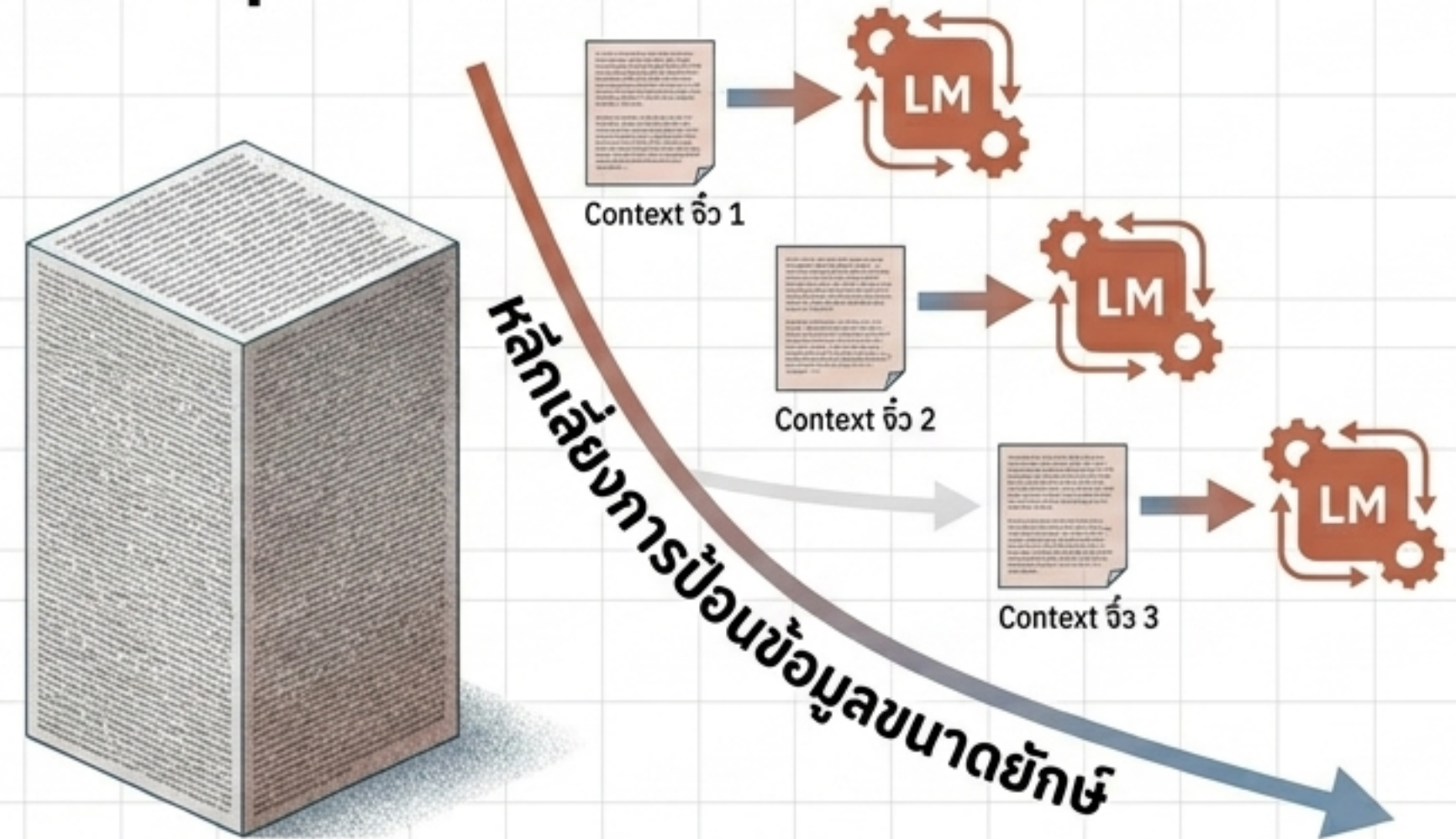
~~\$1.50 - \$2.75~~

ต้นทุน GPT-5 แบบคาดการณ์ที่ 6-11M Tokens

\$0.99

ต้นทุน RLM แท้จริงลดลงจากการใช้ Sub-calls

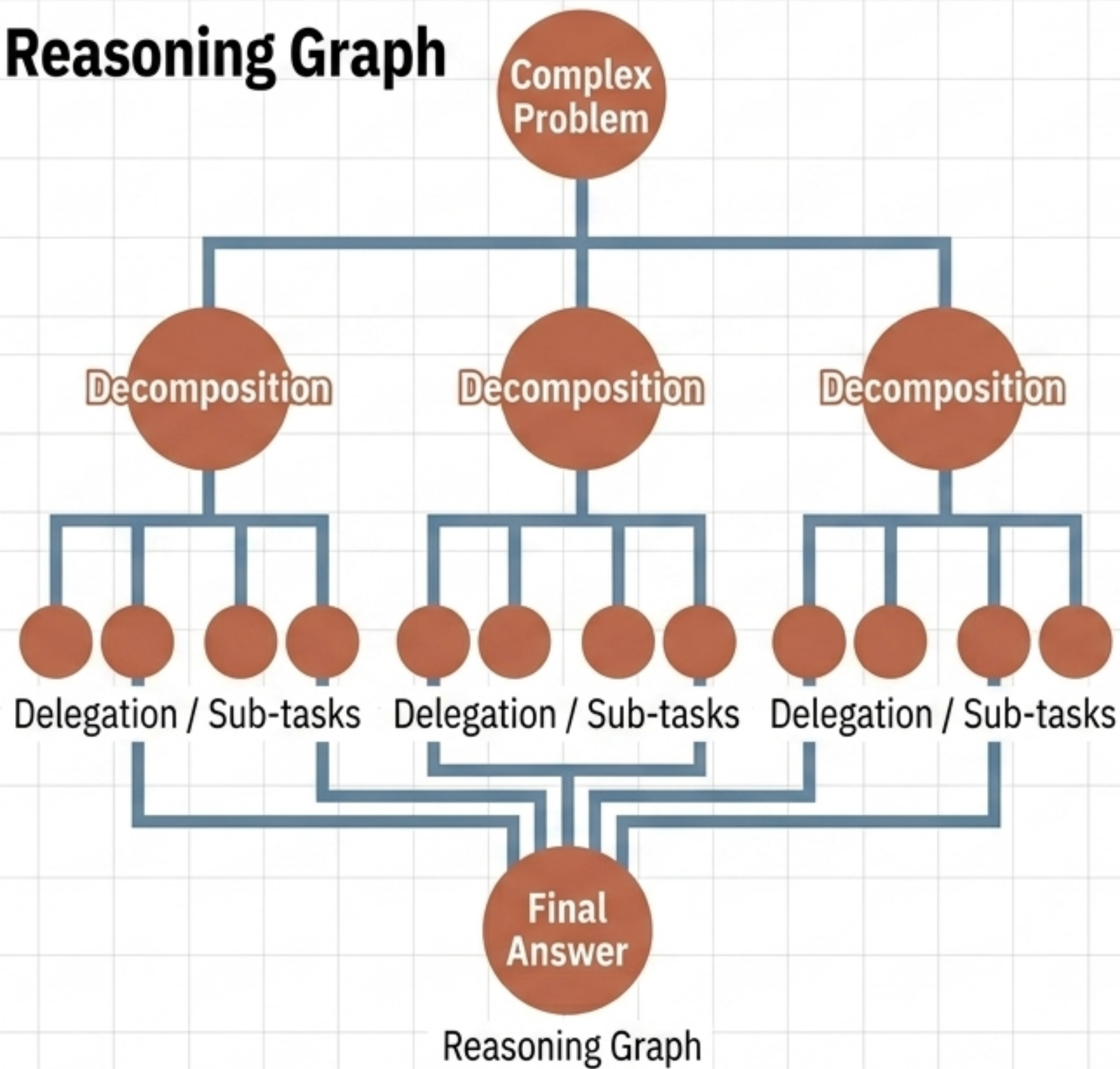
The Explanation



RLM หลักเลี้ยวการป้อนข้อมูลขนาดยักษ์ในทุกๆ Turn แต่เลือกใช้ Context Window ขนาดจิ๋วแบบแยกส่วน ทำให้ต้นทุนรวม (Token Cost) ถูกลงหรือเทียบเท่า ในขณะที่ความแม่นยำสูงกว่า **29%**

ก้าวข้ามแค่ความยาว สู่การให้เหตุผลที่ซับซ้อน (Long-Horizon Reasoning)

Reasoning Graph



LongCoT-mini (math and logic benchmark)

Base GPT-5.2
Solve Rate: [38.7%]

RLM (GPT-5.2) + Decomposition:

[65.6%]

ประสิทธิภาพเพิ่มขึ้น +69.5%

RLM แตกปัญหาคณิตศาสตร์และตรรกะออกเป็น Reasoning Graph และกระจายงานย่อยอย่างเป็นระบบ ทำให้แก้โจทย์ที่ท้าทายที่สุดได้

กายวิภาคของการประมวลผลแบบ RLM

Probe (ตรวจสอบ)

```
# Let's scan the context for clues using keyword searches and show
def find_snippets(keyword, window=200, max_hits=10):
    hits = []
    for i, chunk in enumerate(context):
        start = 0
        while True:
            idx = chunk.lower().find(keyword.lower(), start)
            if idx == -1:
                break
            s = max(0, idx - window)
            e = min(len(chunk), idx + window)
            hit = chunk[s:e].chunk(100)
            if len(hit) == max_hits:
                break
            start = idx + 1
        hits.append(hit)
    return hits

keywords = ["fingering", "pinpoint", "bongoing", "Agoo", "La Union"]
results = []
for k in keywords:
    results.append(find_snippets(k, window=400, max_hits=5))

for k, hits in results.items():
    print(f"--- Keyword: {k} ---")
    if not hits:
        print("No hits")
    for i, hit in enumerate(hits):
        print(f"[{i}] ...{hit}...\n")
```

สแกนและคัดกรองข้อมูล
เบื้องต้นจาก Context
ขนาดมหึมาด้วย Code

Decompose (แยกย่อย)

```
# Let's process questions in batches to be more efficient
def process_batch(questions_batch):
    # Create a prompt for batch processing
    prompt = "Classify each of the following questions into\n"
    prompt += "numeric value, 'entity', 'location', 'desc\n"

    for i, question in enumerate(questions_batch):
        prompt += f"[{i+1}], {question}\n"

    prompt += "\nfor each question, respond with ONLY the c\n"

    result = llm.query(prompt)
    return result.strip().split("\n")
```

กระจายงานวิเคราะห์ไปยัง
บริบทขนาดเล็กผ่าน
Recursive Sub-calls

Stitch (ถักทอ)

```
# Create the final formatted result
formatted_pairs = FF"([pair[0]], {pair[1]})" for pair
final_result = "\n".join(formatted_pairs)

print(f"Total pairs in final result: {len(formatted_pairs)}")
print("First 5 pairs:")
print("\n".join(formatted_pairs[:5]))

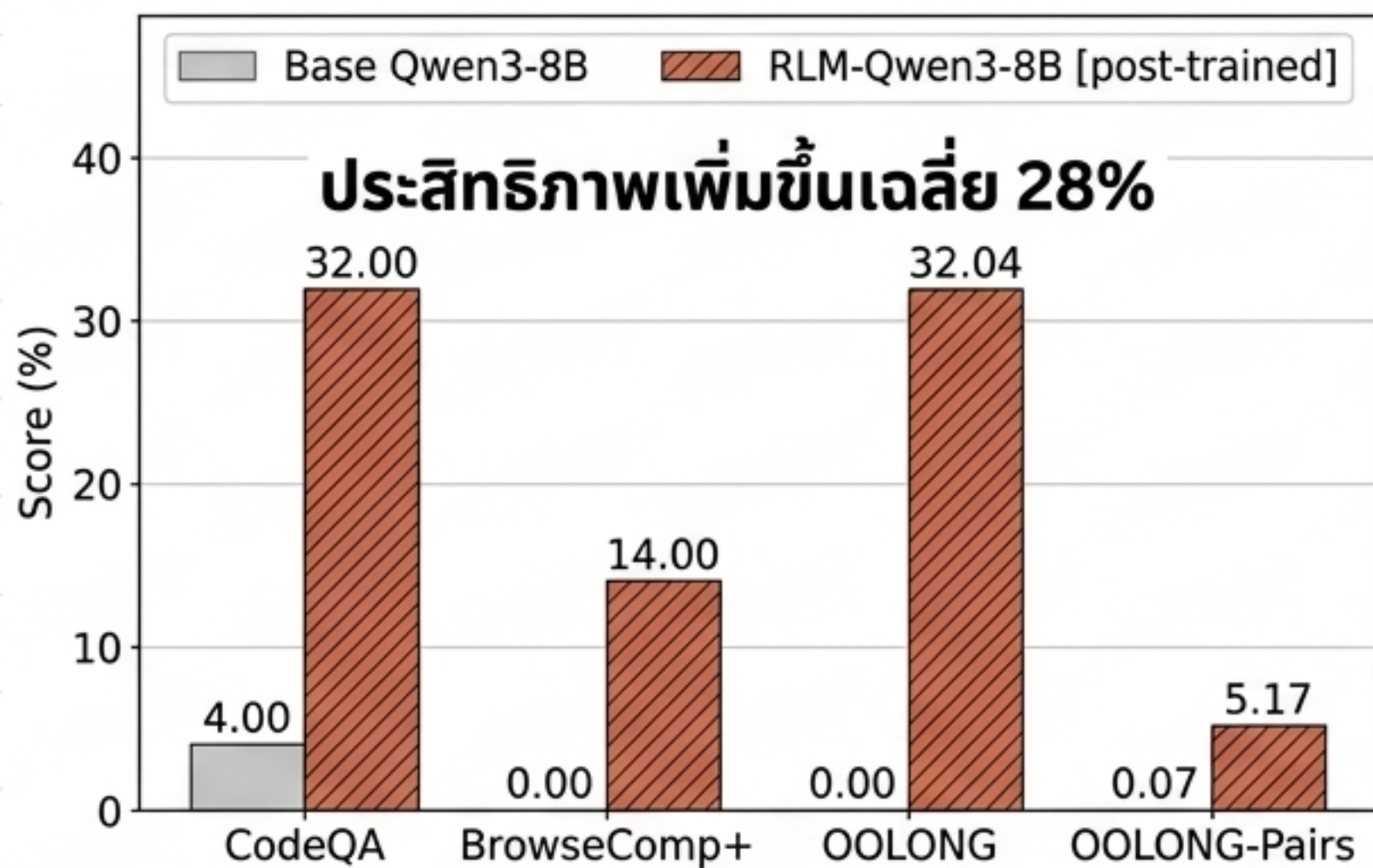
FINAL_VAR(final_result)
```

นำผลลัพธ์ย่อยทั้งหมด
กลับมารวมกันเป็นคำตอบ
สุดท้ายใน REPL

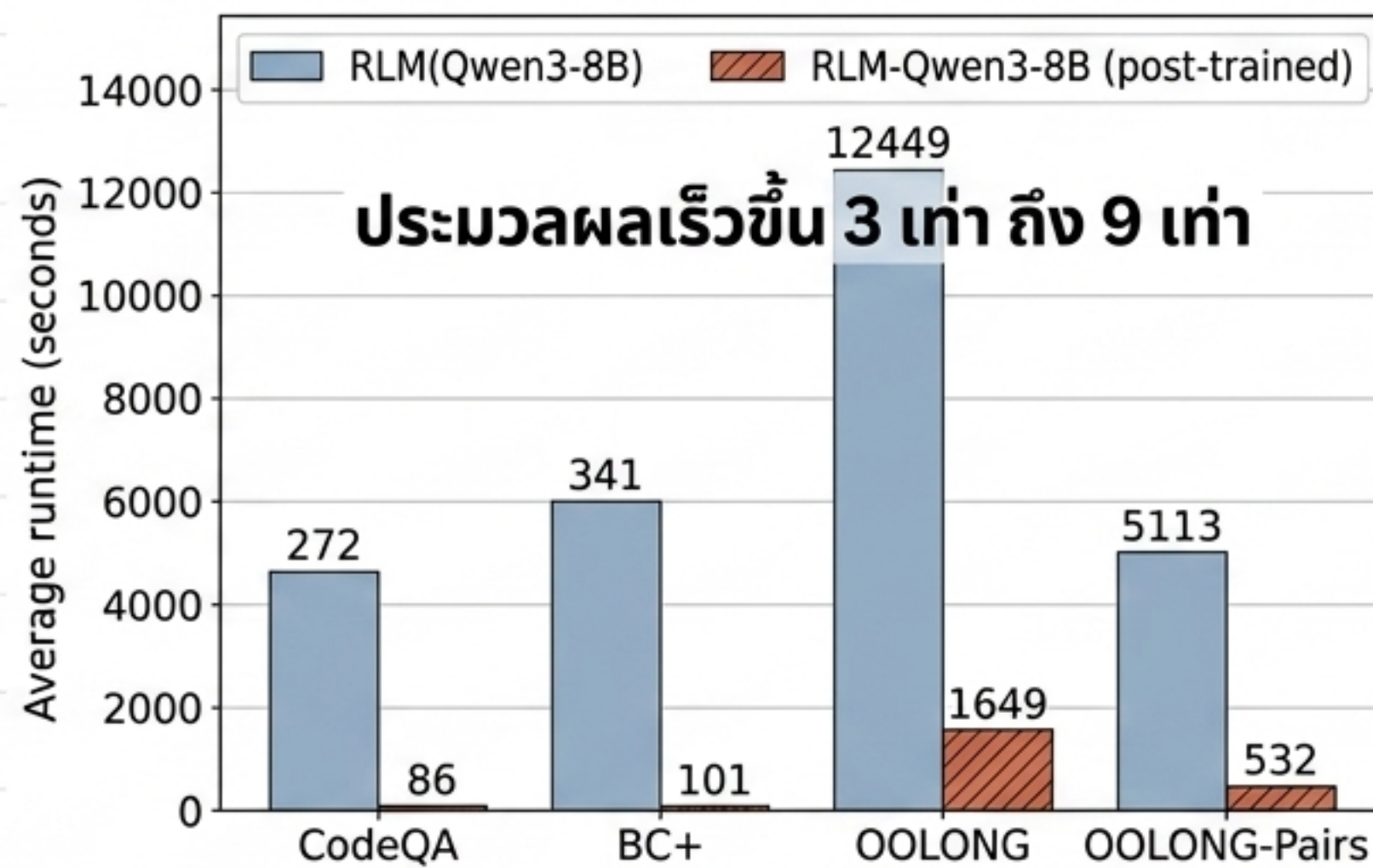
องค์ประกอบของการเขียนโปรแกรมทำให้ RLM ค้นพบเส้นทางสู่คำตอบที่โมเดลปกติมองไม่เห็น

การฝึกสอน RLM โดยกำเนิด: ปลุกพลังโมเดล 8B

Post-training โมเดล Open-source (Qwen3-8B) ด้วย RLM Trajectories เพียง 1,000 ตัวอย่าง



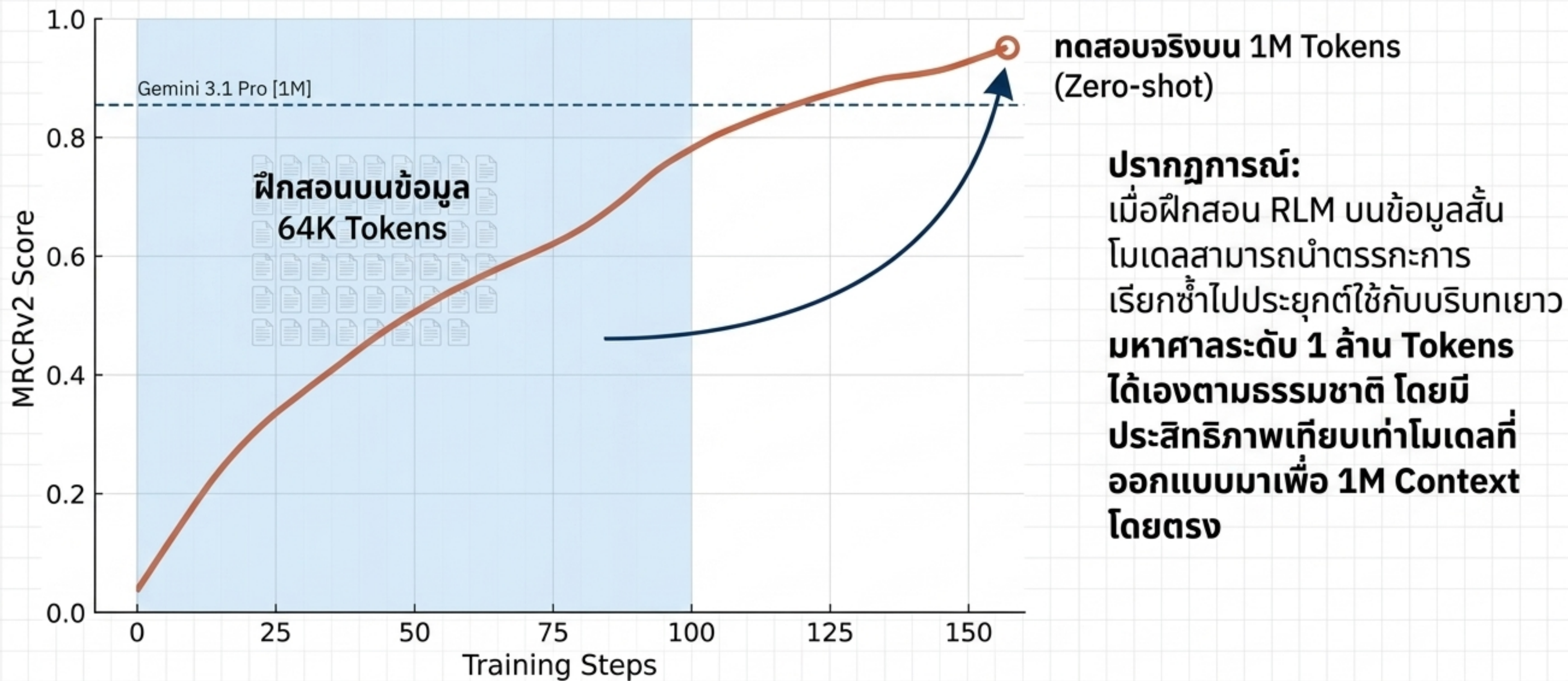
(a) Benchmark scores after fine-tuning (Qwen3-8B RLM)



(b) Average runtime improvement

การสอนให้โมเดลขนาดเล็กคิดแบบ RLM ให้ผลลัพธ์ที่ก้าวกระโดด
ไม่จำเป็นต้องพึ่งพาเพียง Frontier Models

การขยายขีดความสามารถอัตโนมัติ (Length Generalization via RL)



บทสรุปและก้าวต่อไป: แกนใหม่แห่งการสเกล



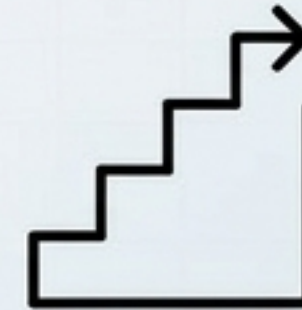
ทะลุขีดจำกัด (Infinite Context)

ย้าย Prompt ออกสู่
Environment ขจัดปัญหา
Context Rot และประมวลผล
ข้อมูลยาวระดับ 10M+ Token
ได้อย่างไร้รอยต่อ



พลังแห่งตรรกะ (Symbolic Recursion)

จัดการข้อมูลที่ซับซ้อน
ระดับสมการเชิงเส้นและ
ควอดราติก (Linear &
Quadratic) ได้อย่างแม่นยำ
ด้วยต้นทุนที่ถูกลง



อนาคตแห่ง AI (New Scaling Axis)

การฝึกสอน Native RLMs
คือแกนใหม่ (New Axis)
ของการขยาย Inference-Time
Compute ที่จะพลิกโฉม
วงการ AI ในยุคถัดไป